Screenshot Memory

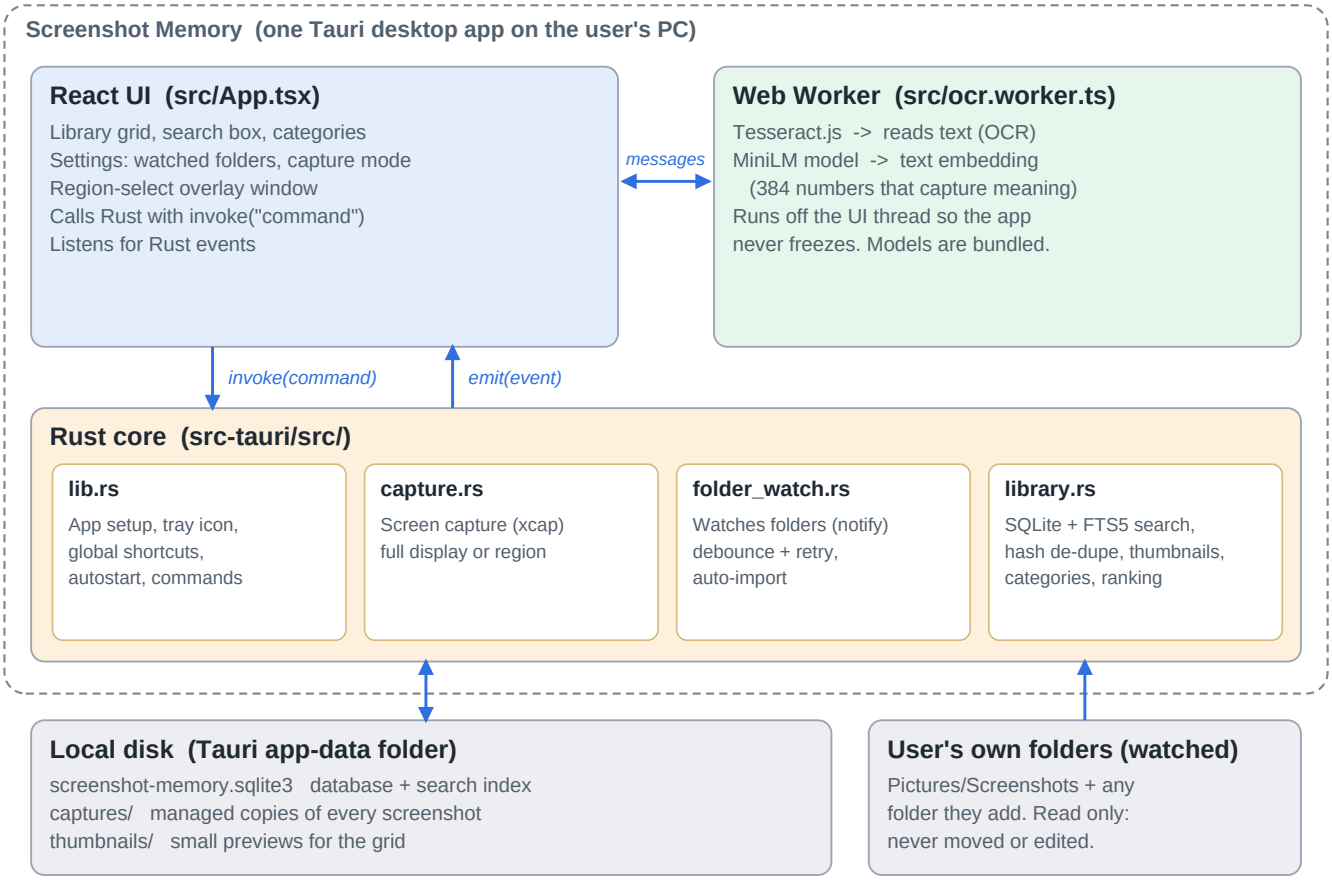
How the app works today, and where we take it next

In one sentence: a Windows desktop app that captures and collects screenshots, reads the text inside them, and lets you find any screenshot later by typing words you remember. Everything runs on the user's PC; no screenshot or text is ever uploaded.

Repo: github.com/bigyangg/capu **Stack:** Tauri 2 (Rust) + React 19 + TypeScript + Vite **Size:** ~3,400 lines of code

1. The big picture

The app has three parts that talk to each other. The **React UI** is what the user sees. Heavy AI work (reading text, understanding meaning) happens in a **Web Worker** so the screen never freezes. Everything that touches files, the screen, or the database is done by the **Rust core**.



Why Tauri? The installer is small (no bundled Chrome like Electron), Rust is fast and safe for file and database work, and the UI is plain React that both of us already know. Users only download an installer; they do not need Rust or Node.

2. Where everything lives

File	Lines	What it does
src/App.tsx	932	The whole UI: library grid, search, categories, settings, region overlay. Runs the OCR queue and talks to Rust and the worker.
src/App.css	-	All styling.
src/ocr.worker.ts	150	Tesseract OCR + MiniLM embeddings. Remote model downloads are switched off; it only loads bundled files.
src-tauri/src/lib.rs	470	Starts the app: plugins, tray icon, global shortcuts, autostart, default Screenshots folder, registers all 28 commands.
src-tauri/src/capture.rs	208	Takes the screenshot (active display, all displays, or a dragged region).
src-tauri/src/folder_watch.rs	151	Watches folders for new images, waits until the file is fully written, then imports it.
src-tauri/src/library.rs	1,435	The heart: SQLite database, migrations, import + de-duplication, thumbnails, search ranking, categories.
src-tauri/capabilities/	-	Security permissions for each window. Keep them minimal.
scripts/copy-ocr-assets.mjs	-	Before every dev/build, copies OCR + ONNX files from node_modules into public/ (that is why those folders are not in git).
public/models/	-	The MiniLM embedding model (23 MB). Committed to git.

3. The database (SQLite)

One file, `screenshot-memory.sqlite3`. Schema changes must be additive (add columns, never drop) so existing users' databases keep working.

Table	Holds
screenshots	One row per image: path, file hash, date, size, status (pending / done / failed), OCR text, thumbnail, category + subcategory (suggested and manual).
screenshot_fts	Full-text search index (FTS5) over the OCR text. This is what makes keyword search instant.
screenshot_embeddings	The 384-number meaning vector for each screenshot, used for "similar meaning" search.
monitored_folders	Folders the user asked us to watch.
settings	Key/value settings (library folder, capture mode, ...).

4. How the UI and Rust talk

Commands (UI asks Rust): `invoke("search_screenshots", {query, embedding})`. There are 28, e.g. list_screenshots, search_screenshots, index_folder, add_monitored_folder, save_ocr_text, capture_screens_now, finish_region_capture. New commands must be registered in lib.rs and their TypeScript types kept in sync.

Events (Rust tells the UI): `capture-created, capture-failed, library-changed, folder-import-failed`. The UI refreshes the library when it hears them.

5. The four main flows

A. Capturing a screenshot

1. User presses **Ctrl+Shift+S** (full display) or **Ctrl+Shift+R** (region). The shortcut works even when the window is hidden in the tray.
2. Rust (capture.rs) grabs the screen with *xcap*. For a region, it opens a full-screen overlay window where the user drags a box; Esc cancels.
3. The image is saved as PNG in the managed `captures/` folder.
4. Rust emits **capture-created**; the UI calls index_folder, so the new row appears with status *pending*.

B. Automatic import (Snipping Tool, or any watched folder)

1. folder_watch.rs listens for new files in Pictures/Screenshots and any folder the user adds.
2. It waits 500 ms after the last change, then tries to read the file. If the file is still being written it retries up to 4 times, waiting longer each time (0.5 s, 1 s, 2 s, 4 s).
3. library.rs hashes the file (SHA-256). If that hash already exists it is a duplicate and is skipped. Otherwise the file is **copied** into the library (the original is never touched) and a thumbnail is made.
4. Rust emits **library-changed**; the UI refreshes.

C. Reading the text (OCR) and understanding it

1. The UI keeps a queue of *pending* screenshots and sends each one to the Web Worker.
2. The worker runs **Tesseract** to read the text, then **MiniLM** turns that text into an embedding (a list of 384 numbers; texts with similar meaning get similar numbers).
3. The UI saves both with save_ocr_text. Rust stores the text in the FTS index, the vector in screenshot_embeddings, and suggests a category from keywords in the text.
4. If OCR fails the row is marked *failed* with the error, so one bad image never blocks the queue. Old rows without embeddings are filled in later.

D. Searching

1. The user types a query. The worker turns the query into an embedding too.
2. Rust scores every screenshot two ways: **keyword match** via FTS5 (score 1.1 + share of words matched) and **meaning match** (cosine similarity of embeddings, counted only if 0.24 or higher).
3. The two scores are added and results are sorted best-first. So exact words win, but "invoice" can still find a screenshot that says "bill".

Categories today: Projects, Finance, Meetings, Development, Reference, each with 3-4 subcategories. Suggestions are simple keyword rules, and the user can override them. The list exists in both library.rs and the UI: change both together.

6. What the app can do today (v0.1.1)

Area	Feature
Capture	Full display or all displays (Ctrl+Shift+S), region select (Ctrl+Shift+R), works from the tray, launch at sign-in option.
Import	Auto-imports from Windows Screenshots folder, add/remove any watched folder, bulk import of existing images, duplicate detection by hash, originals never changed.
Formats	PNG, JPG/JPEG, WebP, BMP, GIF (first frame), TIFF.
Understanding	Local English OCR, local semantic embeddings, automatic category suggestions.
Search	Instant keyword search + meaning-based search combined.
Organize	Categories and subcategories, manual override, thumbnail grid.
Actions	Preview, copy image to clipboard, save a copy elsewhere.
Privacy	100% on device. No network calls for OCR or models.

Honest status

- Frontend build and Rust check both pass. Windows is the only platform we develop on.
- **macOS has never been built or tested.** Do not claim Mac support yet.
- No code signing, no auto-updater, no automated tests, no website release yet.
- OCR is English only. Screenshots with little text are hard to find (no visual search).

7. Rules we do not break

- **Private by default.** Screenshot pixels, OCR text and embeddings never leave the PC. No cloud OCR, no analytics containing content, no model downloads at runtime.
- **Never modify the user's originals.** Import = copy. Never move, rename or delete.
- **Heavy work off the UI thread.** AI in the worker, files and SQL in Rust.
- **Safe SQL and paths.** Always parameterized queries; check every path the UI sends against allowed folders.
- **Minimum permissions** in src-tauri/capabilities/.
- **Additive database migrations only.**
- **No fake buttons.** Every control in the UI must actually work.

8. Roadmap: future features

Ordered so that each phase makes the next one worth doing. Phase 1 is required before anyone outside can install the app from our website.

Phase 1 - Ship it (make it installable from the website)

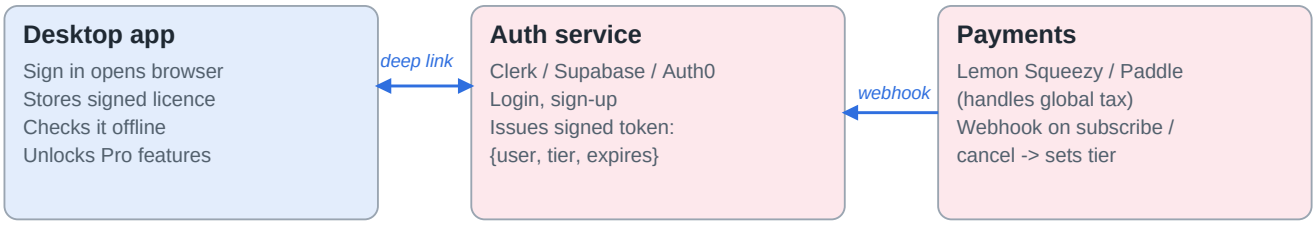
Feature	Why	How
Code signing	Unsigned installers show a scary SmartScreen warning.	Azure Trusted Signing (~$10/month) or an OV/EV certificate; configure in tauri.conf.json.
Installer + releases	One file users download.	`npm run tauri -- build` makes .msi/.exe; host on GitHub Releases or our site.
Auto-updater	Users get fixes without reinstalling.	tauri-plugin-updater with signed update files and a small JSON feed.
Crash/error reports	Know when something breaks.	Opt-in only, never includes screenshot content.
Tests + CI	Two people pushing code = need a safety net.	Rust unit tests for library.rs; GitHub Actions runs build + check on every PR.

Phase 2 - Make it smarter (all still on-device)

Feature	What the user gets	How
Capture context	"The thing I saw in Chrome on Tuesday".	Save active window title + app name + time with each capture. Cheap, huge win.
Visual search	Find charts, photos, UI with no text: "blue dashboard".	CLIP / SigLIP image model on the ONNX runtime we already ship.
Smart extraction	Dates, amounts, links, emails, code pulled out with 1-click actions.	Regex + small local models on OCR text.
Auto captions	A one-line description of every screenshot.	Small vision model (Florence-2 / Moondream).
Ask your screenshots	"What was the API key format from last week?"	Local LLM (llama.cpp) answering from our own search results (RAG).
Smart collections	Auto-groups like "Project X" or "Receipts".	Cluster the embeddings we already store.
More languages	OCR beyond English.	Bundle more Tesseract language packs.

Phase 3 - Accounts, tiers and payments

Only account and licence data goes to our server. Screenshots stay on the PC; this is our main selling point, so the design protects it.



Never sent: screenshots, OCR text, embeddings.
The app verifies the token's signature with a public key, so tiers work offline; it refreshes every few days.

	Free	Pro (idea)
Capture + import	Yes	Yes
Keyword + meaning search	Yes	Yes
Library size	Limited (e.g. 500)	Unlimited
Visual search, captions, smart extraction	-	Yes
Ask your screenshots (local AI)	-	Yes
Extra OCR languages	-	Yes

The Pro features are the heavy AI ones, so charging for them is fair and easy to explain.

Phase 4 - Later

- macOS build (needs Screen Recording permission flow, testing, notarization).
- Optional end-to-end-encrypted sync between the user's own devices.
- Browser extension that captures with the page URL attached.

9. Working together

Setup: install Node.js, Rust (rustup.rs) and Visual Studio C++ Build Tools, then:

```
git clone https://github.com/bigyangg/capu.git
cd capu
npm install
npm run tauri dev
```

- Never push straight to **main**. Make a branch per feature (`git checkout -b visual-search`), push it, open a Pull Request, and the other person reviews it.
- Always `git pull` before starting work.
- Split by area to avoid conflicts, e.g. one person on Phase 1 (signing, updater, CI), the other on Phase 2 features.
- Read CLAUDE.md and README.md in the repo: they hold the rules from section 7.